



Flask

Flask is a lightweight, open-source web framework for Python. It is designed to be simple, flexible, and easy to get started with, making it popular for web applications, REST APIs, prototypes, and microservices.

Key Features

- **Lightweight ("micro" framework)** – Provides core web functionality without forcing a particular project structure.
- **Built-in development server** – Useful for testing during development.
- **URL routing** – Maps URLs to Python functions.
- **Template engine** – Uses Jinja for generating HTML dynamically.
- **Extensible** – Many extensions are available for databases, authentication, forms, and more.

Installation

```
pip install flask
```

Simple Flask Application

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/')  
def home():  
    return "Hello, World!"
```

```
if __name__ == '__main__':  
    app.run(debug=True)
```

Running the Application

```
python app.py
```

By default, the application runs at:

```
http://127.0.0.1:5000
```

Routing Example

```
@app.route('/about')  
def about():  
    return "About Page"
```

Flask installation

First install Flask, you can do so with pip (the Python package manager). Open a terminal and type the command shown:

```
pip install flask
```

That installs the Flask module to your computer. It's recommended that you do that inside a virtual environment.

Flask hello world

We'll create a "Hello world" app for the web. If you load the website url, it will show you the message "Hello world".

Now let's build our app! The first thing to do is to import the Flask module.

```
from flask import Flask
```

Then create the app object.

```
app = Flask(__name__)
```

Start the server with:

```
app.run(host='0.0.0.0', port=5000)
```

This starts the server on your computer at port 5000. If you try a port like 80, it will complain because that's a reserved port.

Finally you'll want a route ('/hello') to display the 'Hello World' message. A route maps a web url (in this case '/hello') to a Python function. Whatever the function returns is shown to the user in the browser.

```
@app.route('/hello')  
def helloIndex():  
    return 'Hello World from Python Flask!'
```

Flask is a micro web framework

Flask is a micro web framework written in Python, meaning it provides only the bare essential tools needed to build a web application while leaving out heavy, pre-configured database layers or authentication tools. This minimalistic approach gives developers complete flexibility to structure their projects and choose their own tools through extensions.

What "Micro" Actually Means

- **No Database Constraints:** Flask lacks a built-in Object-Relational Mapper (ORM), letting you use SQLite, PostgreSQL, or MongoDB freely.
- **No Enforced Structure:** It does not force a rigid directory layout; a simple app can exist entirely within a single Python file.
- **Highly Extensible:** Rather than bundling form validation or login systems, it relies on community extensions like Flask-SQLAlchemy or Flask-WTF.
- **Minimal Dependencies:** The core package relies primarily on the Werkzeug WSGI toolkit and the Jinja template engine.

Core Pillars of the Microframework

Flask's minimal core relies primarily on two key dependencies:

- **Werkzeug:** A WSGI (Web Server Gateway Interface) toolkit used to properly handle HTTP requests, responses, and routing.

Main functions of Werkzeug:

Request handling – Processes HTTP requests coming from the browser.

Response handling – Helps Flask create HTTP responses.

Routing – Matches URLs with the appropriate Flask view function.

Development server – Provides the server used when running Flask during development.

HTTP utilities – Provides tools for handling headers, cookies, status codes, etc.

Debugging – Provides Flask's interactive debugger.

- **Jinja:** A powerful template engine used to generate dynamic HTML content.

Key Benefits

- **No Unnecessary Bloat:** It does not come with a default ORM (Object-Relational Mapper) or form validation layer. You only add what your specific project needs.
- **Maximum Flexibility:** You can swap out components effortlessly, making it highly adaptable for everything from simple APIs to serving machine learning models.
- **Quick Setup:** You can write and run a minimal, functional web application in a single Python file.

Routing

In Flask, **routing** is the process of binding a URL to a specific Python function. When a user visits a URL, Flask matches it to the corresponding function (called a **view function**) and executes it to return a response.

Here is a breakdown of how Flask routing works, from the basics to more advanced techniques.

1. Basic Routing

To create a route, you use the `@app.route()` decorator directly above your view function.

Python

```
from flask import Flask

app = Flask(__name__)

# The root URL
@app.route('/')
def home():
    return "Welcome to the Homepage!"

# A specific URL path
@app.route('/about')
def about():
    return "This is the About page."

if __name__ == '__main__':
    app.run(debug=True)
```

2. Dynamic Routing (Variable Rules)

You can make your URLs dynamic by adding variable sections using `<variable_name>`. Flask passes these variables as arguments to your function.

Python

```
@app.route('/user/<username>')
def show_user_profile(username):
    return f"User: {username}"
```

Converters

By default, variables accept any string without a slash. You can use a **converter** to restrict the data type (e.g., `<converter:variable_name>`).

Converter	Description	Example
string	Accepts any text without a slash (default)	/post/<string:title>
int	Accepts positive integers	/post/<int:post_id>
float	Accepts positive floating-point numbers	/price/<float:amount>
path	Like string but also accepts slashes	/files/<path:subpath>
uuid	Accepts UUID strings	/session/<uuid:id>

Python

```
@app.route('/post/<int:post_id>')
def show_post(post_id):
    # post_id is guaranteed to be an integer
    return f"Post ID: {post_id}"
```

```
# Dynamic routing demo code
```

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
# 1. Standard Route (Homepage)
```

```
@app.route('/')
```

```
def home():
```

```
    return "<h1>Welcome to the Flask Dynamic Routing Demo!</h1><p>Try visiting /user/alex or /product/101</p>"
```

```
# 2. Dynamic Route - Default (String)
```

```
# This accepts any text without a slash (e.g., /user/john, /user/clara)
```

```
@app.route('/user/<username>')
```

```
def show_user_profile(username):
```

```
    # 'username' is passed directly from the URL into the function
```

```
    return f"<h3>User Profile Page</h3><p>Viewing profile for: <strong>{username}</strong></p>"
```

```
# 3. Dynamic Route - Integer Converter
```

```
# The <int:> converter forces the variable to be a positive integer.
```

```
# If someone types /product/abc, Flask will return a 404 error automatically.
```

```
@app.route('/product/<int:product_id>')
```

```
def show_product(product_id):
```

```
    # product_id is guaranteed to be a Python integer here
```

```
    return f"<h3>Product Details</h3><p>Displaying item ID: <code>#{product_id}</code></p>"
```

```
# 4. Dynamic Route - Multiple Variables & Float Converter
```

```
# You can combine multiple variables of different types in a single URL
```

```
@app.route('/discounts/<string:category>/<float:min_price>')
```

```
def show_discounts(category, min_price):
```

```
    return f"<h3>Deals Finder</h3><p>Showing <strong>{category}</strong> items over <strong>${min_price:.2f}</strong></p>"
```

```
if __name__ == '__main__':
```

```
    # Start the application in debug mode
```

```
    app.run(debug=True)
```

How to Test This Program

- 1. Save the code above in a file named app.py.
- 2. Run it in your terminal/command prompt: python app.py.
- 3. Open your browser and try entering these different URLs to see dynamic routing in action:

URL Examples to Try:

URL to enter in Browser	What Flask does behind the scenes	Expected Output on Screen
http://127.0.0.1:5000/user/Sarah	Passes "Sarah" as a string to username.	Viewing profile for: Sarah
http://127.0.0.1:5000/product/455	Passes 455 as an actual integer to product_id.	Displaying item ID: #455
http://127.0.0.1:5000/product/shoes	Fails validation. shoes is not an integer.	404 Not Found
http://127.0.0.1:5000/discounts/electronics/19.99	Passes "electronics" and the float 19.99.	Showing electronics items over \$19.99

3. HTTP Methods

By default, Flask routes only respond to GET requests. To handle other HTTP methods like POST, PUT, or DELETE, pass them in the methods argument.

Python

```
from flask import Flask, request

# Initialize the Flask application
app = Flask(__name__)

# Your login route handling both GET and POST requests
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        return "Processing login credentials..."
    else:
        return "Displaying login form..."

# This block starts the development server
if __name__ == '__main__':
    app.run(debug=True)
```

How to Test Both Methods (GET and POST)

An HTTP **GET** request happens automatically when you visit a URL via a browser. An HTTP **POST** request typically happens when a user submits an HTML form.

Here is how you can test both paths in your code:

Testing the GET path (The else block):

1. Open your web browser (Chrome, Edge, Firefox, etc.).
2. Type **http://127.0.0.1:5000/login** into the address bar and press Enter.
3. Your browser sends a GET request, and you will see: "**Displaying login form...**"

Testing the POST path (The if block):

Since browsers send GET requests by default when you type a URL, you can't test a POST request just by hitting Enter in the address bar. The easiest way to test it without building an HTML form is using your terminal.

4. Keep your Flask server running in the first terminal window.
5. Open a **new, second Command Prompt window**.
6. Type the following curl command (Windows 10/11 has curl built-in) to simulate a form submission:

In DOS prompt
`curl.exe -X POST http://127.0.0.1:5000/login`

7. You will see the terminal print out the response from the POST block: "**Processing login credentials...**"

4. URL Slashes: Unique URLs & Redirection

Flask treats trailing slashes strictly, behaves similarly to how a real file system works: This code block demonstrates how Flask handles **trailing slashes** in URLs, treating them similarly to how file directories work on a computer. Here is exactly what this behavior means and how it works under the hood.

```
from flask import Flask

app = Flask(__name__)

# Case 1: With a trailing slash (Behaves like a folder)
@app.route('/projects/')
def projects():
    return "The projects page"

# Case 2: Without a trailing slash (Behaves like a unique file)
@app.route('/about')
def about():
    return "The about page"

if __name__ == '__main__':
    app.run(debug=True)
```

Steps to Execute

1. Open your **Command Prompt** or **PowerShell** and navigate to your project folder:

DOS

```
cd path\to\your\folder
```

2. Activate your virtual environment if you have one:

DOS

```
venv\Scripts\activate
```

3. Start the Flask server by running:

DOS

```
python app.py
```

to Observe the Trailing Slash Behavior

Open your web browser and test these specific URL variants to see how Flask handles them differently:

Test Group 1: The /projects/ Route (With Slash)

Test A: Go to `http://127.0.0.1:5000/projects/`

Result: The page loads instantly.

Test B: Go to `http://127.0.0.1:5000/projects` (Leave off the last slash)

Result: Look closely at your browser's address bar. Flask automatically redirects you and adds the trailing slash back on, loading the page successfully.

5. URL Building (url_for)

Instead of hardcoding URLs throughout your application, use the `url_for()` function. It generates a URL to a specific function, which prevents broken links if you change your routes later.

```
from flask import Flask, url_for

# Initialize the Flask application
app = Flask(__name__)

# 1. Your homepage route
@app.route('/')
def index():
    # url_for looks for the function named 'show_user_profile' and injects username=
    profile_url = url_for('show_user_profile', username='JohnDoe')

    # We turn the URL into a clickable HTML link for easy testing
    return f"Go to profile: <a href='{profile_url}'>{profile_url}</a>"

# 2. The target route that url_for is mapping to
@app.route('/user/<username>')
def show_user_profile(username):
    return f"Welcome to the profile page of <strong>{username}</strong>!"

if __name__ == '__main__':
    app.run(debug=True)
```

Testing and Understanding url_for

1. Open your browser and navigate to **http://127.0.0.1:5000/**.
2. You will see a webpage displaying: Go to profile: /user/JohnDoe where the path is a clickable link.
3. Click the link. Your browser will immediately navigate to <http://127.0.0.1:5000/user/JohnDoe>.

